



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (Accredited by NBA)

RECORD OF LABORATORY WORK

25CS202 / Application Development Practices

Name of the Student : Rohit GK

Register No. : 25CS612

Year / Branch / Section : I / CSE /

Semester : II

Academic Year : 2025 - 2026



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (Accredited by NBA)

BONAFIDE CERTIFICATE

Name of the Student:

Register No :

Year : Branch : Section : Semester :

Certified that this is the bonafide record of work done by the above student in the
25CS202 / Application Development Practices during the academic year 2025
- 2026.

Faculty Incharge

HOD

Submitted for the End Semester Examination held on

Internal Examiner

External Examiner



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Accredited by NBA)

INDEX

S.NO	LIST OF EXPERIMENTS	PAGE. NO	MARKS	SIGNATURE
1	Create a responsive web page with a registration form using HTMLS, CSS3, and Bootstrap grid system.			
2	Enhance web elements with CSS: Shadows, Rounded Borders, and Multiple Backgrounds.			
3	Implement a CSS grid-based layout to create a responsive web page.			
4	Implement a JavaScript-based form validation for user input fields (email, password, etc.).			
5	Develop a dynamic webpage using JavaScript and DOM manipulation.			
6	Work with APIs and fetch data asynchronously using JavaScript.			
7	Create a TypeScript application demonstrating object-oriented programming concepts.			
8	Write a Linux shell script to automate file management tasks.			
9	Schedule background jobs using Linux cron and batch scripting.			
10	Perform Git version control operations: cloning, committing, branching, and merging.			
11	Collaborate on a project using GitHub: Forking, pull requests, and resolving merge conflicts.			
12	Deploy a static website using GitHub Pages.			



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (Accredited by NBA)

CONTINUOUS EVALUATION SHEET REFERENCES

RUBRICS TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Aim & Algorithm (20)	18-20	17-18	15-16	0-14
Program (30)	27-30	21-26	15-20	0-14
Execution and Result (30)	27-30	21-26	15-20	0-14
Documentation (10)	9-10	7-8	5-6	0-4
Viva (10)	9-10	7-8	5-6	0-4
Overall Marks (100)	90-100	70-89	50-69	0-49



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (Accredited by NBA)

RUBRICS BASED ASSESSMENT SHEET

Name of the Lab:

Reg. No.:

Name of the Student:

Components	EX1	EX2	EX3	EX4	EX5	EX6	EX7	EX8	EX9	EX10	EX11	EX12
Aim& Algorithm (20 Marks)												
Program (30 Marks)												
Execution and Result (30 Marks)												
Documentation (10 Marks)												
Viva (10 Marks)												
TOTAL												
Consolidated Mark (100)												
Faculty Signature												

EXP. NO: 1	Create a responsive web page with a registration form using HTMLS, CSS3, and Bootstrap grid system.
DATE:	

AIM:

To create a responsive web page with a registration form using HTMLS, CSS3, and Bootstrap grid system.

ALGORITHM:

1. Page Setup

- Load HTML document with and language set to English.
- Import Bootstrap CSS (for responsive design and styling).
- Import custom stylesheet IPL.css.

2. Body Layout

- Apply d-flex align-items-center py-4 to center content vertically with padding.
- Create a container to hold the registration card.

3. Card Creation

- Inside the container, a card (ipl-card) is created with padding and max-width.
- The card has a header (ipl-header) containing:
 - A centered title: *"IPL Player Registration"*.
 - An IPL logo image (download.jpg).

4. Form Structure

- A Bootstrap grid (row g-4) organizes form fields into columns.
- Each input field has a label (form-label fw-bold) and a corresponding input element.

5. Form Fields

- **Personal Info:**
 - First Name (text input, required).
 - Last Name (text input, required).
 - Email (email input, required).
 - Age (number input, required).
 - Date of Birth (date input, required).
 - Contact Number (tel input, required).

- Nationality (text input, required).
 - **Player Details:**
 - Player Role (dropdown: Batsman, Bowler, etc.).
 - Bowling Style (dropdown: fast, spin, etc.).
 - Batting Style (dropdown: right/left/ambidextrous).
 - Experience (radio buttons: 7).
 - Previous IPL Teams (dropdown with team names).
 - **Confirmation:**
 - Checkbox to agree to terms & code of conduct (required).
6. **Form Submission**
- A Submit button styled with Bootstrap + custom CSS.
 - On hover, button changes color and slightly enlarges (transform: scale(1.05)).
7. **CSS Styling**
- **Body:** Beige background.
 - **Card (ipl-card):** Cyan background, rounded corners.
 - **Header (ipl-header):** Navy background, yellow text, rounded corners.
 - **Labels:** Styled with italic + bold.
 - **Button:** Blue background, white text, rounded corners, hover effects

SOURCE CODE:

1) HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>IPL Player Registration with BS</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link href="IPL.css" rel="stylesheet">
</head>
<body class="d-flex align-items-center py-4">
```

```
<div class="container">
```

```
  <div class="card ipl-card p-4 p-xl-5 mx-auto" style="max-width: 960px;">
```

```
    <div class="ipl-header d-flex align-items-center justify-content-center">
```

```
      <h2 class="mb-2 text-center flex-grow-1">IPL Player Registration</h2>
```

```
      
```

```
    </div>
```

```
    <form class="row g-4 p-3">
```

```
      <div class="col-md-6">
```

```
        <label for="fullName" class="form-label fw-bold">First Name <span class="text-  
danger">*</span></label>
```

```
        <input type="text" class="form-control" id="fullName" placeholder="Enter your First  
Name" required>
```

```
      </div>
```

```
      <div class="col-md-6">
```

```
        <label for="fullName" class="form-label fw-bold">Last Name <span class="text-  
danger">*</span></label>
```

```
        <input type="text" class="form-control" id="fullName" placeholder="Enter your Last  
Name" required>
```

```
      </div>
```

```
      <div class="col-md-6">
```

```
        <label for="email" class="form-label fw-bold">Email<span class="text-  
danger">*</span></label>
```

```
        <input type="email" class="form-control" placeholder="Enter your Email" id="email"  
required>
```

```
      </div>
```

```
      <div class="col-md-3">
```

```
        <label for="age" class="form-label fw-bold">Age <span class="text-  
danger">*</span></label>
```

```
        <input type="number" class="form-control" id="age" placeholder="Enter your Age"  
maxlength="2" required>
```


</div>

<div class="col-md-3">

<label for="dob" class="form-label fw-bold">Date of Birth *</label>

<input type="date" class="form-control" required>

</div>

<div class="col-md-6">

<label for="phone" class="form-label fw-bold">Contact Number *</label>

<input type="tel" class="form-control" id="phone" maxlength="10" placeholder="Enter your Contact Number" required>

</div>

<div class="col-md-6">

<label for="nation" class="form-label fw-bold">Nationality *</label>

<input type="text" class="form-control" id="nation" placeholder="Enter your Nationality" required>

</div>

<div class="col-md-12">

<label for="playerRole" class="form-label fst-italic fw-bold">Player Role</label>

<select class="form-select" id="role" required>

<option>-- Select your Role --</option>

<option>Batsman</option>

<option>Bowler</option>

<option>All-Rounder</option>

<option>Wicket-Keeper</option>

</select>

</div>

<div class="col-md-6">

```
<label for="bowlhand" class="form-label fst-italic fw-bold">Bowling Style</label>
<select class="form-select" id="bowlstyle" required>
  <option>-- Select your style --</option>
  <option>N/A (Pure Batter)</option>
  <option>Right-Handed Fast</option>
  <option>Right-Handed Medium</option>
  <option>Right-Handed Leg-Spin</option>
  <option>Right-Handed Off-Spin</option>
  <option>Left-Handed Fast</option>
  <option>Left-Handed Medium</option>
  <option>Left-Handed Leg-Spin</option>
  <option>Left-Handed Off-Spin</option>
</select>
</div>
```

```
<div class="col-md-6">
  <label for="bathand" class="form-label fst-italic fw-bold">Batting Style</label>
  <select class="form-select" id="batstyle" required>
    <option>-- Select your style --</option>
    <option>Right-Handed</option>
    <option>Left-Handed</option>
    <option>Ambidextrous</option>
  </select>
</div>
```

```
<div class="col-md-12">
  <label for="experience" class="form-label fst-italic fw-bold">Experience</label>
  <div class="d-flex gap-5 pt-2">
    <div class="form-check">
      <input class="form-check-input" type="radio" name="experience">
      <label class="form-check-label"> Less than 3 Years</label>
    </div>
  </div>
</div>
```

```
<div class="form-check">
  <input class="form-check-input" type="radio" name="experience">
  <label class="form-check-label"> 3 to 5 years</label>
</div>

<div class="form-check">
  <input class="form-check-input" type="radio" name="experience">
  <label class="form-check-label"> 5-7 years</label>
</div>

<div class="form-check">
  <input class="form-check-input" type="radio" name="experience">
  <label class="form-check-label"> More than 7 years</label>
</div>
</div>
</div>

<div class="col-md-12">
  <label for="team" class="form-label fst-italic fw-bold">Previous IPL Teams (if
any)</label>
  <select class="form-select" id="team" required>
    <option>-- Select your Team --</option>
    <option>N/A</option>
    <option>Chennai Super Kings</option>
    <option>Mumbai Indians</option>
    <option>Delhi Capitals</option>
    <option>Gujarat Giants</option>
    <option>Lucknow Super Giants</option>
    <option>Sunrisers Hyderabad</option>
    <option>Royal Challengers Bengaluru</option>
    <option>Rajasthan Royals</option>
    <option>Kolkata Knight Riders</option>
    <option>Punjab Kings</option>
  </select>
</div>
```

```

<div class="col-12 mt-4">
  <div class="form-check">
    <input class="form-check-input" type="checkbox" id="termsCheck" required>
    <label class="form-check-label fst-italic fw-semibold" for="termsCheck">
      I confirm that the information provided is accurate and I agree to the IPL player
      terms & code of conduct. <span class="text-danger">*</span>
    </label>
    <div class="invalid-feedback">You must accept to register.</div>
  </div>
</div>

<div class="col-12">
  <button type="submit" class="col-12 d-flex align-items-center justify-content-
  center">Submit</button>
</div>
</form>
</div>
</div>
</body>
</html>

```

2) CSS Code:

```

body{
  background-color: beige;
}

.ipl-card{
  background-color: cyan;
  border-radius: 35px;
  border-color: 10px solid white;
}

```

```
.ipl-header{
    background: navy;
    border-radius: 25px;
    height: 100px;

    h2{
        text-align: center;
        color: yellow;
    }

    img{
        border-radius: 25px;
        padding-right: 7px;
    }
}

.form-label i {
    color: #1e3c72;
    width: 22px;
    margin-right: 6px;
}

button{
    border: none;
    background-color: blue;
    color: white;
    border-radius: 25px;
    height: 50px;
    transition: background-color 0.3s ease,
        color 0.3s ease,
        transform 0.3s ease;
}
```

```
button:hover{  
  transition: 0.2s ease;  
  background-color: white;  
  color: black;  
  transform: scale(1.05);  
}
```

OUTPUT:

IPL Player Registration



First Name *

Enter your First Name

Last Name *

Enter your Last Name

Email*

Enter your Email

Age *

Enter your Age

Date of Birth *

dd-mm-yyyy

Contact Number *

Enter your Contact Number

Nationality *

Enter your Nationality

Player Role

-- Select your Role --

Bowling Style

-- Select your style --

Batting Style

-- Select your style --

Experience

☐ Less than 3 Years ☐ 3 to 5 years ☐ 5-7 years ☐ More than 7 years

Previous IPL Teams (if any)

-- Select your Team --

☐ I confirm that the information provided is accurate and I agree to the IPL player terms & code of conduct. *

Submit

RESULT:

Hence the output for the sample code using HTML, CSS and Bootstrap has been displayed.

EXP. NO: 2	Enhance web elements with CSS: Shadows, Rounded Borders, and Multiple Backgrounds.
DATE:	

AIM:

To enhance web elements with CSS: Shadows, Rounded Borders, and Multiple Backgrounds.

ALGORITHM:

1. Page Initialization
 - Load HTML document with and set language to English.
 - Define metadata (charset, viewport) and page title ("Shadow").
 - Apply internal CSS styles.
2. Body Layout
 - Use flexbox (display: flex; align-items: center; justify-content: center;) to center content both vertically and horizontally.
 - Set body height to full viewport (100vh).
3. Profile Card (.shadow)
 - Create a div with class shadow.
 - Apply styles:
 - Dotted border with rounded corners (border-radius: 50px).
 - Different border colors (top = blue, bottom = red).
 - Box shadow (3px 3px 30px blue) for glowing effect.
 - Flexbox column layout for stacking elements vertically.
 - Bold, large font with a modern typeface.
4. Card Content
 - Name Label (.pa1)
 - Displays "Rohit G K".
 - Styled with red text shadow (3px 3px 3px rgb(255,0,0)).
 - Profile Image
 - Displays "My Photo.jpg".
 - Image height = 200px, rounded corners (border-radius: 50px).
 - About Me Label (.pa2)
 - Displays "About Me..".

- (Note: .pa2 has no CSS defined, so it inherits default styles).
 - Paragraph (.p1)
 - Contains personal details: location, date/time, favorites, hobbies, personality.
 - Styled with medium font size and centered text.
5. User Interaction
- Page is static (no JavaScript).
 - User simply views the profile card with styled text, image, and shadow effects

SOURCE CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shadow</title>
  <style>
    body{
      display: flex;
      align-items: center;
      justify-content: center;
      height: 100vh;
    }
    .shadow{
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
      border-style: dotted;
      height: auto;
      width: 600px;
      font-weight: bolder;
      font-size: xx-large;
```

```
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
border-radius: 50px;
border-top-color: blue;
border-bottom-color: red;
box-shadow: 3px 3px 30px blue;
}
```

```
.pa1{
  text-shadow: 3px 3px 3px rgb(255, 0, 0);
}
```

```
.p1{
  font-size: medium;
  text-align: center;
}
```

```
img{
  height: 200px;
  border-radius: 50px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="shadow">
```

```
<label class="pa1">Rohit G K </label>
```

```
<br>
```

```
<label class="pa2">About Me..</label>
```

```
<p class="p1">
```

Hello, I am from Coimbatore South, Tamil Nadu, India.
It's currently a pleasant Thursday afternoon, 12 March 2026.

- Favorite Food: Rasam With Potato Fry

- Favorite Car: Koenigsegg Agera RS

- Hobby: Biking

- Personality: I enjoy learning, exploring new ideas, and expressing myself in creative ways.

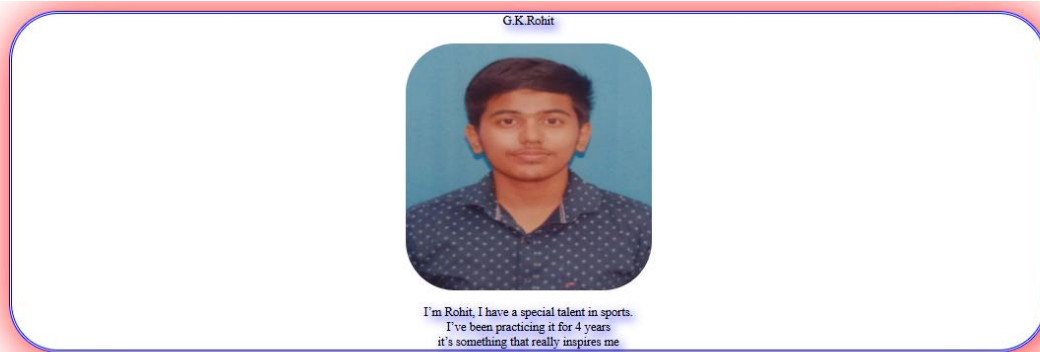
</p>

</div>

</body>

</html>

OUTPUT:



RESULT:

From the source code, the web elements are enhanced using CSS.

EXP. NO: 3	Implement a CSS grid-based layout to create a responsive web page.
DATE:	

AIM:

To implement a CSS grid-based layout to create a responsive web page.

ALGORITHM:

1. Page Initialization

- Load HTML document with and set language to English.
- Define metadata (charset, viewport) and page title ("CSS Grid").
- Import Bootstrap CSS for responsive grid system.
- Apply internal CSS for font family and size.

2. Container Setup

- Create a container-fluid to allow full-width layout.
- Inside it, define a row with padding (p-5).

3. Row Structure

- The row is divided into multiple columns using Bootstrap's grid system.

4. Header Section

- col-12: Full-width column.
- Centered content (d-flex align-items-center justify-content-center).
- Bordered with green (border border-3 border-success).
- Displays label: "**Rohit G K**".

5. Left Column

- col-md-4: Occupies 4/12 columns on medium+ screens.
- Centered content with flexbox.
- Contains an unordered list (ul) with items: **Hello, World, Universe**.
- List style removed (list-style-type: none).

6. Right Column

- col-md-8: Occupies 8/12 columns on medium+ screens.
- Centered content with flexbox.
- Contains a paragraph (p) with personal details:

- Location, date/time.
- Favorite food, car, hobby.
- Personality description.

7. Footer Section

- col-12: Full-width column.
- Centered content with flexbox.
- Displays label: “**Rohit SKCET** ©”.
- Styled with green border.

SOURCE CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSS Grid</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body{
      font-family: 'Gill Sans', 'Gill Sans MT', Calibri, 'Trebuchet MS', sans-serif;
      font-size: 24px;
    }
  </style>
</head>
<body>
  <div class="container-fluid">
    <div class="row p-5">
      <div class="col-12 d-flex align-items-center justify-content-center border border-3
border-success">
        <label>Rohit G K</label>
      </div>
```

```
<div class="col-md-4 d-flex align-items-center justify-content-center border border-3 border-success">
```

```
<ul style="list-style-type: none;">
```

```
<li>Hello</li>
```

```
<li>World</li>
```

```
<li>Universe</li>
```

```
</ul>
```

```
</div>
```

```
<div class="col-md-8 d-flex align-items-center justify-content-center border border-3 border-success">
```

```
<p>Hello, I am from Coimbatore South, Tamil Nadu, India. It's currently a pleasant Thursday afternoon, 12 March 2026.
```

```
<br>- Favorite Food: Rasam With Potato Fry
```

```
<br>- Favorite Car: Koenigsegg Agera RS
```

```
<br>- Hobby: Biking
```

```
<br>- Personality: I enjoy learning, exploring new ideas, and expressing myself in creative ways.</p>
```

```
</div>
```

```
<div class="col-12 d-flex align-items-center justify-content-center border border-3 border-success">
```

```
<label>Rohit || SKCET &copy;</label>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

OUTPUT:

Rohit G K	
Hello World Universe	Hello, I am from Coimbatore South, Tamil Nadu, India. It's currently a pleasant Thursday afternoon, 12 March 2026. - Favorite Food: Rasam With Potato Fry - Favorite Car: Koenigsegg Agera RS - Hobby: Biking - Personality: I enjoy learning, exploring new ideas, and expressing myself in creative ways.
Rohit SKCET ©	

RESULT:

Responsive web page is created by implementing CSS Grid-based layout.

EXP. NO: 4	Implement a JavaScript-based form validation for user input fields (email, password, etc.).
DATE:	

AIM:

To Implement a JavaScript-based form validation for user input fields (email, password, etc.).

ALGORITHM:

1. Page Setup

- Load HTML document with <!DOCTYPE html>.
- Define metadata (charset, viewport) and page title ("Form Validation").
- Apply CSS styles to center the form and style the fieldset.

2. Form Layout

- Create a fieldset with a legend titled "Form Validation".
- Inside the form:
 - Input field for **Contact** (id="phone").
 - Input field for **Email** (id="email").
 - A **Validate** button that triggers the validate() function.
 - A element (id="res") to display the result.

3. Validation Function (validate())

- Retrieve values from input fields:
 - $ph \leftarrow$ phone number.
 - $em \leftarrow$ email address.
- Define regex patterns:
 - $pat = ^{[0-9]\{10\}}\$$ → ensures phone has exactly 10 digits.
 - $pat1 = ^{[A-Za-z0-9]+\@[a-z](\.[a-z]\{2,\})+\$}$ → ensures email format is valid.

4. Validation Logic

- If **phone OR email** does not match their respective patterns:
 - If phone is invalid:
 - Show alert: *"Enter the contact correctly!!"*.
 - Set result text = *"Invalid Contact"*.
 - Else (email invalid):

- Show alert: *"Enter the email correctly!!"*.
- Set result text = *"Invalid Email"*.
- Else (both valid):
- Show alert: *"Details Validated Successfully"*.
- Set result text = *"Valid Details"*.

5. Result Display

- Update the element with the validation result.

SOURCE CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Validation</title>
  <style>
    body{
      display: flex;
      align-items: center;
      justify-content: center;
    }

    form{
      text-align: center;
    }

    fieldset{
      height: 200px;
      width: 400px;
    }

    legend{
```

```
        text-align: center;
    }
</style>
<script>
    function validate(){
        var ph=document.getElementById("phone").value;
        var em=document.getElementById("email").value;
        var pat=/^[0-9]{10}$/;
        var pat1=/^[A-Za-z0-9]+@[a-z](\.[a-z]{2,})+$/;
        if(!pat.test(ph) || !pat1.test(em)){
            if(!pat.test(ph)){
                alert("Enter the contact correctly!!");
                var res="Invalid Contact";
            }
            else{
                alert("Enter the email correctly!!");
                var res="Invalid Email";
            }
        }
        else{
            alert("Details Validated Successfully");
            var res="Valid Details";
        }
        document.getElementById("res").innerHTML=res;
    }
</script>
</head>
<body>
    <fieldset>
        <legend><h1>Form Validation</h1></legend>
        <form>
            <label>Contact: </label><input type="text" id="phone"><br><br>
```

```
<label>Email: </label><input type="text" id="email"><br><br>
<button onclick="validate()">Validate</button>
<p id="res"> </p>
</form>
</fieldset>
</body>
</html>
```

OUTPUT:

Form Validation

Contact:

Email:

127.0.0.1:5500 says

Details Validated Successfully

OK

Contact:

Email:

Form Validation

Contact:

Email:

127.0.0.1:5500 says

Enter the contact correctly!!

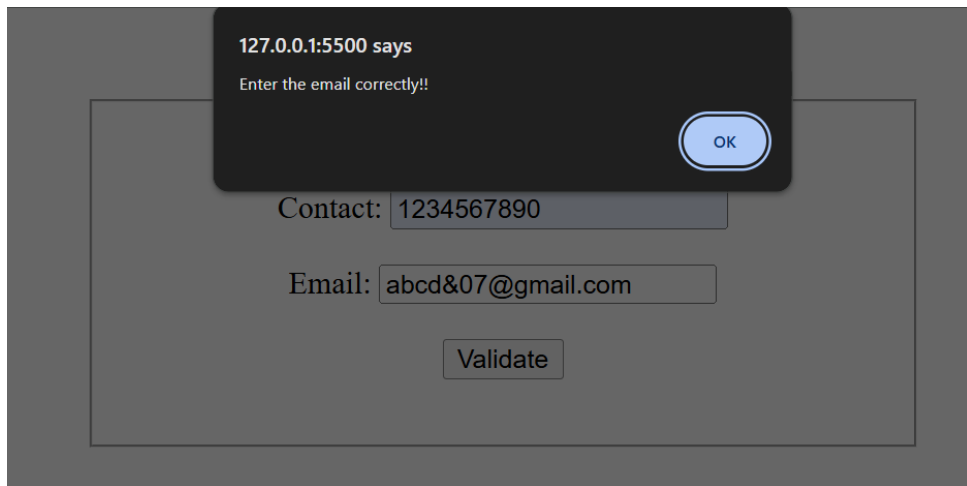
Contact:

Email:

Form Validation

Contact:

Email:



RESULT:

Responsive web page is created by implementing CSS Grid-based layout.

EXP. NO: 5	Develop a dynamic webpage using JavaScript and DOM manipulation.
DATE:	

AIM:

To develop a dynamic webpage using JavaScript and DOM manipulation.

ALGORITHM:

1. Page Setup

- Load HTML document with <!DOCTYPE html>.
- Import external CSS file (Calculator.css).
- Set page title to "Calculator".

2. Heading

- Display a heading label: "**Calculator**" styled with large bold font.

3. Input Section

- Two text input fields:
 - Num-1 (id="n1")
 - Num-2 (id="n2")
- Both inputs allow user to enter numbers.

4. Operation Buttons

- Four buttons are displayed:
- **Add** → calls sum()
- **Subtract** → calls sub()
- **Multiply** → calls mul()
- **Divide** → calls div()

5. Result Display

- A label: "**Result:**"
- Another label (id="res2") where the output will be shown.

6. JavaScript Functions

- **sum()**
 - Get values from n1 and n2.
 - Convert to integers (parseInt).
 - Display result as "Sum : a+b".

- **sub()**
 - Get values from n1 and n2.
 - Display result as "Difference : a-b".
- **mul()**
 - Get values from n1 and n2.
 - Display result as "Product : a*b".
- **div()**
 - Get values from n1 and n2.
 - If n2 != 0, display "Quotient : a/b".
 - Else, show alert "Division by Zero is not possible".

7. CSS Styling

- **Body:** Centered layout with red border.
- **Heading (.hh):** Large bold font.
- **Labels (.lab):** Margin spacing.
- **Input Section (.inp):** Flexbox centered with bold font.
- **Operation Buttons (#b1, #b2, #b3, #b4):**
 - Each button has unique colors and font size.
- **Result (.res, #res2):**
- Styled with padding and larger font.

SOURCE CODE:

1) HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="Calculator.css">
  <title>Calculator</title>
</head>
<body>
  <div class="heading">
```



```
<label class="hh">Calculator</label>

</div>

<div class="inp">

    <label for="n1" class="lab">Num-1 : </label><input type="text" class="lab" id="n1"
size="30"><br><br>

    <label for="n2" class="lab">Num-2 : </label><input type="text" class="lab" id="n2"
size="30"><br><br>

</div>

<div class="operation">

    <input type="button" id="b1" value="Add" onclick="sum()">
    <input type="button" id="b2" value="Subtract" onclick="sub()">
    <input type="button" id="b3" value="Multiply" onclick="mul()">
    <input type="button" id="b4" value="Divide" onclick="div()">

</div>

<br><br><br>

<label class="res">Result: </label><label id="res2"></label>

<script>

    function sum(){
        let a=parseInt(document.getElementById("n1").value);
        let b=parseInt(document.getElementById("n2").value);
        document.getElementById("res2").innerText="Sum : "+(a+b);
    }

    function sub(){
        let a=parseInt(document.getElementById("n1").value);
        let b=parseInt(document.getElementById("n2").value);
        document.getElementById("res2").innerText="Difference : "+(a-b);
    }

    function mul(){
        let a=parseInt(document.getElementById("n1").value);
        let b=parseInt(document.getElementById("n2").value);
```

```
        document.getElementById("res2").innerText="Product : "+(a*b);
    }

    function div(){
        let a=parseInt(document.getElementById("n1").value);
        let b=parseInt(document.getElementById("n2").value);
        if(b!=0){
            document.getElementById("res2").innerText="Quotient : "+(a/b);
        }
        else{
            alert("Division by Zero is not possible");
        }
    }
</script>
</body>
</html>
```

2) CSS Code:

```
body{
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    border: 5px solid red;
    margin-top: 100px;
}

.hh{
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    font-weight: 900;
    font-size: 64px;
}
```

```
.lab{  
  margin: 10px;  
}
```

```
.inp{  
  display: flex;  
  margin: 10px;  
  align-items: center;  
  justify-content: center;  
  margin-top: 20px;  
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
  font-weight: 600;  
}
```

```
.operation{  
  border: none;  
  #b1{  
    color: red;  
    background-color: aqua;  
    font-size: 20px;  
  }
```

```
  #b2{  
    color: red;  
    background-color: yellow;  
    font-size: 20px;  
  }
```

```
  #b3{  
    color: white;  
    background-color: blue;
```

```
        font-size: 20px;
    }

    #b4{
        color: yellow;
        background-color: green;
        font-size: 20px;
    }
}

.res{
    padding-bottom: 30px;
    font-size: x-large;
    font-family: 'Gill Sans', 'Gill Sans MT', Calibri, 'Trebuchet MS', sans-serif;
}

#res2{
    padding-bottom: 20px;
    font-size: 24px;
}
```

OUTPUT:

Calculator

Num-1: Num-2:

Add Subtract Multiply Divide

Result:

RESULT:

A Dynamic web page is created using JavaScript and DOM manipulation.

EXP. NO: 6	Work with APIs and fetch data asynchronously using JavaScript.
DATE:	

AIM:

To work with APIs and fetch data asynchronously using JavaScript.

ALGORITHM:

- 1. Start the program**
Load the HTML page in a browser.
- 2. Display button interaction**
Wait for the user to click the **DISPLAY** button.
- 3. Trigger function**
When the button is clicked, call the display () function.
- 4. Fetch data from API**
 - Send an asynchronous request to the URL:
<https://jsonplaceholder.typicode.com/users>
 - Wait for the response.
- 5. Convert response to JSON**
Parse the response into a JavaScript object (data).
- 6. Initialize output string**
Create an empty string variable output.
- 7. Iterate through user data**
For each user in the data array:
 - Extract the name field.
 - Extract the city field from address.city.
 - Append both values to the output string with formatting.
- 8. Update HTML content**
Insert the output string into the element with id "result".
- 9. End program**
The names and cities of all users are displayed on the webpage.

SOURCE CODE:

```
<!DOCTYPE html>

<html>

  <body>

    <H1> API INTERFACE PROGRAM</H1>

    <br><br>

    <button onclick="display()">DISPLAY</button>

    <br>

    <p id="result"></p>

    <script>

      async function display()

      {

        let info=await fetch("https://jsonplaceholder.typicode.com/users");

        let data=await info.json();

        let output="";

        for(let i=0;i<data.length;i++)

        {

          output+="<br>Name:"+ data[i].name+ "<br>City:"+ data[i].address.city+"<br>";

        }

        document.getElementById("result").innerHTML=output;

      }

    </script>

  </body>

</html>
```

OUTPUT:

API INTERFACE PROGRAM

DISPLAY

Name:Leanne Graham
City:Gwenborough

Name:Ervin Howell
City:Wisokyburgh

Name:Clementine Bauch
City:McKenziehaven

Name:Patricia Lebsack
City:South Elvis

Name:Chelsey Dietrich
City:Roscoeview

Name:Mrs. Dennis Schulist
City:South Christy

Name:Kurtis Weissnat
City:Howemouth

Name:Nicholas Runolfsdottir V
City:Aliyaview

Name:Glenna Reichert
City:Bartholomebury

Name:Clementina DuBuque
City:Lebsackbury

RESULT:

JavaScript enables smooth interaction with APIs through asynchronous data fetching. Using `fetch()` and promises, data can be retrieved and processed without blocking the UI. This approach enhances responsiveness and demonstrates real-time integration of external data into web applications.

EXP. NO: 7	Create a TypeScript application demonstrating object-oriented programming concepts.
DATE:	

AIM:

To create a TypeScript application demonstrating object-oriented programming concepts.

ALGORITHM:

1. Define Person Class

1. Properties:
 - name → public
 - age → private
 - city → public
2. Constructor initializes name, age, and city.
3. Methods:
4. setAge(a:number) → updates private age.
5. getAge() → returns private age.
6. display() → prints name, age (via getter), and city.

2. Define Employee Class (extends Person)

1. Inherits all properties and methods of Person.
2. Adds new properties:
 - empld → public
 - empSal → public
3. Constructor:
 - Calls super(n,a,c) to initialize inherited properties.
 - Initializes empld and empSal.
4. Overrides display():
5. Calls super.display() to print Person details.
6. Prints empld and empSal.

3. Object Creation

1. Create an Employee object:

```
obj = new Employee("Rohit", 18, "Coimbatore", 1001, 15000);
```

This initializes:

2. name = "Rohit"
3. age = 18
4. city = "Coimbatore"
5. empld = 1001
6. empSal = 15000

4. Method Execution

- Call obj.display().
- Execution flow:
- Employee.display() runs.
- Calls Person.display() → prints name, age, city.
- Then prints employee ID and salary.

SOURCE CODE:

```
class Person{  
    public name:string;  
    private age:number;  
    public city:string;  
  
    constructor(n:string, a:number, c:string){  
        this.name=n;  
        this.age=a;  
        this.city=c;  
    }  
  
    setAge(a:number):void{  
        this.age=a;  
    }  
    getAge():number{  
        return this.age;  
    }  
}
```

```
display():void{
    console.log("Name:",this.name,"\nAge:",this.getAge(),"\nCity:",this.city);
}
}
```

```
class Employee extends Person{
    public empld:number;
    public empSal:number;

    constructor(n:string, a:number, c:string, ei:number, es:number){
        super(n,a,c);
        this.empld=ei;
        this.empSal=es;
    }

    display():void{
        super.display();
        console.log("Employee ID:",this.empld,"\nSalary:",this.empSal);
    }
}
```

```
var obj=new Employee("Rohit",18,"Coimbatore",1001,15000);
obj.display();
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

C:\Program Files\nodejs\node.exe --experimental-network-inspection .\TypeScript\OOPS.ts
Name: Aswath
Age: 18
City: Coimbatore
Employee ID: 1001
Salary: 15000
```

RESULT:

A TypeScript application is created demonstrating object-oriented programming concepts.